



COMPUTAÇÃO EM NUVEM

AULA 6

Prof. Maíke Cristian Rebelo de Lima



CONVERSA INICIAL

Atualmente, os bancos de dados em nuvem estão no centro das transformações digitais que tantas empresas buscam para otimizar custos e melhorar a eficiência. A migração de bancos de dados para a nuvem não só elimina a necessidade de investimentos em infraestrutura física, mas também proporciona escalabilidade e resiliência em níveis que antes eram difíceis de alcançar com soluções locais.

No contexto de modelos de serviços em nuvem, os bancos de dados podem ser oferecidos como parte de uma plataforma PaaS (Plataforma como Serviço), em que os provedores cuidam da maioria das tarefas de gerenciamento, como backups, segurança e atualizações, permitindo que as empresas foquem em usar os dados de forma estratégica. Exemplos disso incluem o Amazon RDS, o Google Cloud SQL e o Azure SQL Database.

Um aspecto crítico a considerar é o modelo de responsabilidade compartilhada. Nos bancos de dados em nuvem, a segurança dos dados é uma responsabilidade conjunta entre o provedor e o cliente. O provedor cuida da segurança da infraestrutura e oferece ferramentas para proteção de dados, como criptografia e firewalls. Já o cliente é responsável pelo controle de acesso, configuração de permissões e segurança dos dados. Esse modelo requer que as empresas tenham políticas bem definidas de Gerenciamento de Identidade e Acesso (IAM) para que apenas usuários autorizados acessem os dados.

A conformidade regulatória também é um fator-chave, principalmente com legislações como o GDPR e a LGPD. Muitos provedores oferecem bancos de dados com configurações de conformidade já implementadas, ajudando as empresas a garantirem que suas práticas de armazenamento e segurança estejam em conformidade com os requisitos legais.

Além disso, a escalabilidade é um dos maiores atrativos dos bancos de dados em nuvem. Com serviços escaláveis, uma empresa pode ajustar o desempenho de acordo com o aumento (ou diminuição) da demanda, pagando apenas pelo que usa. Em vez de adquirir novos servidores ou expandir a infraestrutura local, um banco de dados em nuvem pode facilmente escalar para atender a picos de demanda.

Por fim, o acesso e a flexibilidade são fundamentais para muitas organizações que buscam uma estratégia de dados centralizada. Bancos de



dados em nuvem permitem que os dados sejam acessíveis de qualquer lugar, mantendo a segurança e o controle de acesso.

TEMA 1 – BANCO DE DADOS RELACIONAL: AMAZON WEB SERVICES – AWS

O tipo de banco de dados mais comum em uso são os bancos de dados relacionais. São usados nas mais variadas formas de aplicações, sejam redes sociais, *e-commerce*, blogs ou em aplicações empresariais de grande porte. Os bancos de dados relacionais mais populares são o MySQL, PostgreSQL, Microsoft SQL Server e os Bancos de Dados Oracle. Os bancos de dados relacionais fornecem uma interface gráfica amigável para manipulação dos dados, utilizando o *Structured Query Language* (SQL). Um banco de dados consiste em uma ou mais tabelas, as quais consistem em um conjunto de linhas e colunas, similar com planilhas do Excel. Uma coluna do banco de dados contém um atributo específico do registro, como o nome, o endereço e o número de telefone de uma pessoa. Cada atributo é atribuído a um tipo de dados como texto, número ou data, e o mecanismo do banco de dados rejeitará entradas inválidas. Na Figura 1 a seguir, demonstramos uma tabela chamada “Students”.

Figura 1 – Exemplo de tabela de banco de dados relacional

StudentID	FirstName	LastName	Gender	Age
1001	Joe	Dusty	M	29
1002	Andrea	Romanov	F	20
1003	Ben	Johnson	M	30
1004	Beth	Roberts	F	30

Fonte: Codd, 1985.

Essa tabela de exemplo possui quatro registros, com cada registro representando um estudante individual. Cada estudante possui um campo StudentID, que geralmente é um número único por estudante. Um número único que identifica cada estudante pode ser chamado de “chave primária”.

Um registro em uma tabela pode se relacionar com um registro em outra tabela, referenciando a chave primária de um registro. Esse ponteiro ou referência é chamado de “chave estrangeira”. Por exemplo, uma tabela Grades



que registra as notas de cada estudante teria sua própria chave primária e uma coluna adicional conhecida como chave estrangeira, que se refere à chave primária do registro do estudante. Referenciando as chaves primárias de outras tabelas, bancos de dados relacionais minimizam a duplicação de dados em tabelas associadas. Nos bancos de dados relacionais, é importante observar que a estrutura da tabela (como o número de colunas e o tipo de dados de cada coluna) deve ser definida antes que os dados sejam adicionados à tabela.

Um banco de dados relacional pode ser categorizado como um sistema de Processamento de Transações On-line (OLTP) ou Processamento Analítico On-line (OLAP), dependendo de como as tabelas estão organizadas e de como a aplicação utiliza o banco de dados relacional. OLTP refere-se a aplicações orientadas a transações que frequentemente escrevem e alteram dados (por exemplo, entrada de dados e *e-commerce*). OLAP é tipicamente usado em data warehouses e refere-se a relatórios ou análises de grandes conjuntos de dados. Grandes aplicações frequentemente têm uma mistura de bancos de dados OLTP e OLAP.

O Amazon Relational Database Service (Amazon RDS) simplifica significativamente a configuração e manutenção de bancos de dados OLTP e OLAP. O Amazon RDS oferece suporte a seis populares motores de banco de dados relacionais: MySQL, Oracle, PostgreSQL, Microsoft SQL Server, MariaDB e Amazon Aurora. Também podemos optar por executar praticamente qualquer motor de banco de dados usando instâncias do Amazon Elastic Compute Cloud (Amazon EC2) com Windows ou Linux e gerenciar a instalação e administração por conta própria.

1.1 Amazon Relational Database Service (Amazon RDS)

O Amazon RDS é um serviço que simplifica a configuração, operação e escalabilidade de um banco de dados relacional na AWS. Com o Amazon RDS, podemos focar mais na aplicação e no esquema, enquanto o Amazon RDS assume tarefas comuns como backups, aplicação de patches, escalabilidade e replicação.

O Amazon RDS ajuda a simplificar a instalação do software de banco de dados e o provisionamento da capacidade de infraestrutura. Em poucos minutos, o Amazon RDS pode lançar um dos diversos motores de banco de dados populares que estão prontos para começar a receber transações SQL. Após o



lançamento inicial, o Amazon RDS facilita a manutenção contínua, automatizando tarefas administrativas comuns de forma recorrente.

Com o Amazon RDS, podemos acelerar seus prazos de desenvolvimento e estabelecer um modelo operacional consistente para gerenciar bancos de dados relacionais. Por exemplo, o Amazon RDS facilita a replicação dos seus dados para aumentar a disponibilidade, melhorar a durabilidade ou escalar além de uma única instância de banco de dados para cargas de trabalho com grande volume de leitura.

O Amazon RDS expõe um endpoint de banco de dados ao qual o software cliente pode se conectar e executar comandos SQL. O Amazon RDS não fornece acesso ao shell das Instâncias de Banco de Dados (DB) e restringe o acesso a determinados procedimentos e tabelas do sistema que exigem privilégios avançados. Com o Amazon RDS, podemos usar, em geral, as mesmas ferramentas para consultar, analisar, modificar e administrar o banco de dados. Por exemplo, as ferramentas de extração, transformação e carga (ETL) e de relatórios atuais podem se conectar aos bancos de dados do Amazon RDS da mesma forma, com os mesmos drivers, sendo muitas vezes necessário apenas mudar o hostname na string de conexão para reconfigurar.

1.2 Banco de dados (instâncias)

O serviço Amazon RDS oferece uma interface de programação de aplicações (API) que permite criar e gerenciar uma ou mais instâncias de banco de dados (DB Instances). Uma DB Instance é um ambiente de banco de dados isolado, implantado em segmentos privados da sua rede na nuvem. Cada DB Instance executa e gerencia um motor de banco de dados popular, comercial ou de código aberto, em seu nome. O Amazon RDS atualmente oferece suporte aos seguintes motores de banco de dados: MySQL, PostgreSQL, MariaDB, Oracle, SQL Server e Amazon Aurora.

Podemos iniciar uma nova DB Instance chamando a API `CreateDBInstance` ou usando o Console de Gerenciamento da AWS. Instâncias de banco de dados existentes podem ser modificadas ou redimensionadas usando a API `ModifyDBInstance`. Uma DB Instance pode conter vários bancos de dados diferentes, que criamos e gerenciamos dentro da própria instância, executando comandos SQL no endpoint do Amazon RDS.



Esses diferentes bancos de dados podem ser criados, acessados e gerenciados com as mesmas ferramentas e aplicativos de cliente SQL que já utilizamos.

Os recursos de computação e memória de uma DB Instance são determinados pela sua classe de instância. Podemos escolher a classe de instância que melhor atende às nossas necessidades de computação e memória. A variedade de classes vai desde uma db.t2.micro com 1 CPU virtual (vCPU) e 1 GB de memória, até uma db.r3.8xlarge com 32 vCPUs e 244 GB de memória. À medida que suas necessidades mudam, podemos alterar a classe da instância, e o Amazon RDS migrará seus dados para uma instância maior ou menor, conforme necessário.

Independentemente da classe de instância escolhida, também podemos controlar o tamanho e as características de desempenho do armazenamento utilizado.

O Amazon RDS oferece suporte a uma grande variedade de motores, versões e combinações de recursos. Podemos consultar a documentação do Amazon RDS para verificar o suporte a recursos específicos. Muitos recursos e configurações comuns são gerenciados usando grupos de parâmetros de banco de dados (DB parameter groups) e grupos de opções de banco de dados (DB option groups). Um grupo de parâmetros de banco de dados atua como um contêiner para valores de configuração do motor que podem ser aplicados a uma ou mais DB Instances. Podemos alterar o grupo de parâmetros de uma instância existente, mas será necessário reiniciar a instância. Um grupo de opções de banco de dados atua como um contêiner para os recursos do motor, que, por padrão, é vazio. Para habilitar recursos específicos de um motor de banco de dados (por exemplo, Oracle Statspack ou SQL Server Mirroring), criamos um novo grupo de opções de banco de dados e configuramos as definições conforme necessário.

1.3 Benefícios operacionais

O Amazon RDS aumenta a confiabilidade operacional dos seus bancos de dados ao aplicar um modelo de implantação e operação muito consistente. Esse nível de consistência é alcançado em parte ao limitar os tipos de alterações que podem ser feitas na infraestrutura subjacente e com o uso extensivo de automação. Por exemplo, no Amazon RDS, não é possível usar o Secure Shell (SSH) para fazer login na instância de host e instalar um software personalizado.



No entanto, podemos nos conectar usando ferramentas de administração SQL ou usar grupos de opções de banco de dados (DB option groups) e grupos de parâmetros de banco de dados (DB parameter groups) para alterar o comportamento ou a configuração de recursos de uma DB Instance. Se desejamos controle total do sistema operacional (OS) ou precisamos de permissões elevadas para execução, devemos considerar a instalação de nosso banco de dados no Amazon EC2 em vez do Amazon RDS.

O Amazon RDS foi projetado para simplificar as tarefas comuns necessárias para operar um banco de dados relacional de forma confiável. É útil comparar as responsabilidades de um administrador ao operar um banco de dados relacional em seu data center, no Amazon EC2 ou com o Amazon RDS. No Quadro 1 a seguir, demonstramos as responsabilidades entre o cliente e o *provider*.

Quadro 1 – Matriz de responsabilidade entre os tipos de banco de dados

Responsabilidade	Banco de Dados On Premisse	Banco de Dados em uma Instância EC2	Banco de Dados no Amazon RDS
Otimização da Aplicação	Cliente	Cliente	Cliente
<i>Scaling</i>	Cliente	Cliente	AWS
Alta Disponibilidade	Cliente	Cliente	AWS
Backup	Cliente	Cliente	AWS
Patches no Banco de Dados	Cliente	Cliente	AWS
Instalação de Software	Cliente	Cliente	AWS
Patches do Sistema Operacional	Cliente	Cliente	AWS
Instalação do Sistema Operacional	Cliente	AWS	AWS
Manutenção do Servidor	Cliente	AWS	AWS



Responsabilidade com a <i>Stack</i>	Cliente	AWS	AWS
Energia e Resfriamento	Cliente	AWS	AWS

Fonte: Amazon.

1.4 Amazon Aurora

O Amazon Aurora oferece tecnologia de banco de dados comercial de nível empresarial, ao mesmo tempo que oferece simplicidade e economia de um banco de dados de código aberto. Isso é conseguido através do redesenho dos componentes internos do MySQL para adotar uma abordagem mais orientada a serviços.

Assim como outros mecanismos Amazon RDS, o Amazon Aurora é um serviço totalmente gerenciado, é MySQL – compatível imediatamente e fornece maior confiabilidade e desempenho em implantações padrão do MySQL. Amazon Aurora pode oferecer até cinco vezes mais desempenho do MySQL sem exigir alterações na maioria dos nossos aplicativos web existentes. Podemos usar o mesmo código, ferramentas e aplicativos que usamos com nossos bancos de dados MySQL existentes com Amazon Aurora.

Ao criar uma instância do Amazon Aurora pela primeira vez, criamos um cluster de banco de dados. Um cluster de banco de dados tem uma ou mais instâncias e inclui um volume de cluster que gerencia os dados dessas instâncias. Um volume de cluster do Amazon Aurora é um volume de armazenamento de banco de dados virtual que abrange múltiplas zonas de disponibilidade, com cada zona de disponibilidade tendo uma cópia dos dados do cluster. O cluster de banco de dados Amazon Aurora consiste em dois tipos diferentes de instâncias:

- instância primária – é a instância principal, que suporta cargas de trabalho de leitura e gravação. Ao modificar seus dados, estamos modificando a instância primária. Cada cluster de banco de dados do Amazon Aurora tem uma instância primária;
- réplica do Amazon Aurora – é uma instância secundária que oferece suporte apenas a operações de leitura.



Cada cluster de banco de dados pode ter até 15 réplicas do Amazon Aurora, além do primeiro exemplo. Ao usar várias réplicas do Amazon Aurora, podemos distribuir a carga de trabalho de leitura entre diversas instâncias, aumentando o desempenho. Também podemos provisionar nosso Amazon Aurora Réplicas em múltiplas zonas de disponibilidade para aumentar a disponibilidade do nosso banco de dados.

1.5 Opções de armazenamento

O Amazon RDS é desenvolvido usando o Amazon Elastic Block Store (Amazon EBS) e permite que selecionemos a opção de armazenamento correta com base em nossos requisitos de desempenho e custo. Dependendo no mecanismo de banco de dados e na carga de trabalho, podemos escalar até 4 a 6 TB em armazenamento provisionado e até 30.000 IOPS. O Amazon RDS oferece suporte a três tipos de armazenamento: magnético e de uso geral (unidade de estado sólido [SSD]) e IOPS provisionados (SSD).

1.6 Backup e restauração

O Amazon RDS fornece um modelo operacional consistente para procedimentos de backup e recuperação nos diferentes mecanismos de banco de dados. O Amazon RDS fornece dois mecanismos para backup o banco de dados: backups automatizados e instantâneos manuais. Usando uma combinação de ambas as técnicas, podemos projetar um modelo de recuperação de backup para proteger os dados do seu aplicativo.

Cada organização normalmente definirá um Objetivo de Ponto de Recuperação (RPO) e um Tempo de Recuperação Objetivo (RTO) para aplicações importantes com base na criticidade da aplicação e no expectativas dos usuários. É comum que os sistemas empresariais tenham um RPO medido em minutos e um RTO medido em horas ou até dias, enquanto algumas aplicações críticas têm tolerâncias muito mais baixas.

RPO é definido como o período máximo de perda de dados aceitável em caso de falha ou incidente. Por exemplo, muitos sistemas fazem backup dos logs de transações a cada 15 minutos para permitir e minimizar a perda de dados em caso de exclusão acidental ou falha de hardware.



O RTO é definido como a quantidade máxima de tempo de inatividade que é permitido recuperar de backup e para retomar o processamento. Especialmente para grandes bancos de dados, pode levar horas para restaurar a partir de um backup completo. No caso de uma falha de hardware, podemos reduzir seu RTO para minutos fazendo failover para um nó secundário. Devemos criar um plano de recuperação que, no mínimo, permite recuperar de um backup recente.

1.7 Backup automatizado

Um backup automatizado é um recurso do Amazon RDS que rastreia continuamente alterações e retrocessos no nosso banco de dados. O Amazon RDS cria um snapshot do volume de armazenamento da sua instância de banco de dados, fazendo backup de toda a instância do banco de dados e não apenas de bancos de dados individuais. Podemos definir o backup período de retenção quando criamos uma instância de banco de dados. Um dia de backups será retido por padrão, mas podemos modificar o período de retenção até um máximo de 35 dias. É preciso termos em mente que, quando excluimos uma instância de banco de dados, todos os snapshots de backup automatizados são excluídos e não podem ser recuperados. Os backups manuais, entretanto, não são excluídos.

Os backups automatizados ocorrerão diariamente durante uma janela de manutenção configurável de 30 minutos chamada de “janela de backup”. Os backups automatizados são mantidos por um número configurável de dias, chamado de “período de retenção de backup”. Podemos restaurar sua instância de banco de dados para qualquer horário específico durante esse período de retenção, criando uma nova instância de banco de dados.

1.8 Recuperação

O Amazon RDS permite que recuperemos seu banco de dados rapidamente, seja executando backups automatizados ou instantâneos de banco de dados manuais. Não podemos restaurar um snapshot de banco de dados para um instância de banco de dados existente; uma nova instância de banco de dados é criada quando restauramos. Quando restauramos uma instância do banco de dados, apenas o parâmetro de banco de dados padrão e os grupos de



segurança estão associados ao restaurado exemplo. Assim que a restauração for concluída, deveremos associar qualquer parâmetro de banco de dados personalizado ou grupos de segurança usados pela instância da qual restauramos. Ao usar backups automatizados, o Amazon RDS combina os backups diários realizados durante seus backups predefinidos.

TEMA 2 – BANCO DE DADOS RELACIONAL: OUTROS PROVEDORES

2.1 Azure SQL Database

O Azure SQL Database é o serviço de banco de dados relacional gerenciado da Microsoft. Ele foi projetado para oferecer alta disponibilidade, segurança robusta e escalabilidade flexível, eliminando a necessidade de gerenciamento de infraestrutura física. O serviço é compatível com SQL Server e se integra profundamente ao ecossistema Azure.

2.1.1 Estrutura de dados

No Azure SQL Database, os dados são organizados em tabelas que contêm colunas e linhas, permitindo modelagem relacional tradicional. Cada tabela utiliza chaves primárias para identificar registros de forma única e chaves estrangeiras para relacionar tabelas diferentes. Isso garante a consistência e minimiza a redundância de dados.

2.2 Principais funcionalidades

2.2.1 Modelos de instância

- **Data Transaction Units (DTU):** combina desempenho de CPU, memória e IOPS em um único medidor, ideal para cargas de trabalho previsíveis.
- **vCore (Virtual Core):** proporciona maior flexibilidade, permitindo ajustar CPU, memória e armazenamento separadamente.
- **Instâncias gerenciadas:** compatíveis com o SQL Server local para simplificar migrações e modernizações.



2.2.2 Escalabilidade

- **Vertical:** aumenta ou diminui os recursos de uma instância conforme necessário.
- **Horizontal:** bancos de dados elásticos permitem compartilhar recursos entre múltiplos bancos.
- **Automação:** ajustes automáticos baseados em carga de trabalho.

2.2.3 Alta disponibilidade

- Georreplicação ativa para recuperar rapidamente de desastres.
- Failover automático para instâncias secundárias em diferentes regiões.

2.2.4 Segurança

- Criptografia: proteção de dados em trânsito e em repouso.
- Autenticação centralizada: integração com Azure Active Directory (AAD).
- Detecção de ameaças: monitoramento proativo para identificar atividades suspeitas.

2.2.5 Backup e recuperação

- Backups automáticos configuráveis com restauração pontual.
- Retenção de backups por até 35 dias.

2.2.6 Integração com o Ecossistema Azure

- Power BI: relatórios e visualizações interativos.
- Azure Data Factory: pipelines de dados para movimentação e transformação de dados.
- Azure Machine Learning: uso de dados para criação de modelos de IA.

2.2.7 Benefícios operacionais

O Azure SQL Database permite que as empresas se concentrem no uso estratégico dos dados, enquanto a Microsoft cuida de tarefas administrativas, como backups, atualizações de segurança e escalabilidade. A compatibilidade



com SQL Server torna o Azure SQL Database uma escolha natural para empresas que já utilizam produtos Microsoft.

2.2.8 Comparação com outros serviços

- **Amazon RDS:** oferece suporte a mais motores de banco de dados, mas o Azure SQL Database possui integração nativa mais profunda com ferramentas Microsoft.
- **Google Cloud SQL:** focado em startups e no ecossistema do Google Cloud, enquanto o Azure SQL é mais robusto para empresas que já utilizam o Azure.

2.2.9 Restrições

Algumas funcionalidades avançadas de SQL Server podem não estar disponíveis dependendo do modelo de instância escolhido. Para controle total, pode-se optar por hospedar o SQL Server em uma máquina virtual do Azure.

2.3 Google Cloud SQL

O Google Cloud SQL é um serviço gerenciado de banco de dados relacional que oferece suporte aos motores MySQL, PostgreSQL e SQL Server. Ele é projetado para simplificar a configuração, operação e escalabilidade de bancos de dados relacionais, permitindo que as empresas se concentrem mais nas aplicações e nos dados do que na infraestrutura.

2.3.1 Estrutura de dados

No Google Cloud SQL, um banco de dados é composto por uma ou mais tabelas que contêm linhas e colunas, semelhante a planilhas. Cada coluna tem um tipo de dado definido, como texto, número ou data. Chaves primárias identificam de forma única cada registro, e chaves estrangeiras permitem relacionar tabelas diferentes, minimizando a duplicação de dados.

2.4 Principais funcionalidades

2.4.1 Suporte a múltiplos motores

- **MySQL:** ideal para aplicações web e sistemas transacionais.



- **PostgreSQL:** suporte a extensões e análises avançadas.
- **SQL Server:** para workloads empresariais com dependências de SQL Server.

2.4.2 Escalabilidade

- Ajuste automático de recursos para lidar com aumentos de carga.
- Suporte a réplicas de leitura para distribuir cargas de leitura intensiva.

2.4.3 Alta disponibilidade e recuperação

- Failover automático e replicação regional para alta disponibilidade.
- Backups automáticos configuráveis e restauração pontual para recuperação de desastres.

2.4.4 Integração com o ecossistema Google Cloud

- **BigQuery:** conexão nativa para análises avançadas.
- **Firebase:** ideal para aplicações móveis e web.
- **Looker:** relatórios e visualizações em tempo real.

2.4.5 Segurança e conformidade

- Criptografia de dados em trânsito e repouso.
- Gerenciamento de acesso granular com Google Cloud IAM.
- Conformidade com regulamentos como GDPR e PCI DSS.

2.4.6 Ferramentas de otimização

- Insights de consultas com recomendações automáticas para melhorar o desempenho.
- Monitoramento e alertas por meio do Google Cloud Monitoring.

2.4.7 Benefícios operacionais

O Google Cloud SQL elimina a necessidade de configurar servidores manualmente. Ele fornece endpoints para que aplicativos clientes se conectem



usando ferramentas SQL padrão, como MySQL Workbench, pgAdmin ou SQL Server Management Studio.

2.4.8 Comparação com outros serviços

- **Amazon RDS:** suporte a mais motores (incluindo MariaDB e Amazon Aurora), mas sem a integração nativa com ferramentas do Google Cloud, como Firebase e BigQuery.
- **Azure SQL Database:** mais bem integrado ao ecossistema Microsoft, enquanto o Cloud SQL é mais forte no ecossistema de startups e empresas que já usam Google Cloud.

2.4.9 Restrições

Recursos avançados do sistema operacional não são acessíveis, pois o Cloud SQL é um serviço gerenciado. Para controle total, recomenda-se usar instâncias de banco de dados em máquinas virtuais no Compute Engine.

No Quadro 2 seguir, apresentamos um comparativo entre os provedores e os bancos de dados relacionais.

Quadro 2 – Comparação entre os Bancos SQL entre os provedores

Aspecto	Amazon RDS (AWS)	Azure SQL Database	Google Cloud SQL
Modelos Suportados	MySQL, PostgreSQL, MariaDB, Oracle, SQL Server, Amazon Aurora	SQL Server, Banco relacional PaaS (DTU/vCore), Instâncias Gerenciadas	MySQL, PostgreSQL, SQL Server
Escalabilidade	Vertical (mudança de instância), Réplicas de leitura, Multi-AZ	Vertical e horizontal (bancos elásticos)	Vertical (ajuste automático de recursos)
Alta Disponibilidade	Suporte Multi-AZ, failover automático	Failover automático e geo-replicação ativa	Failover automático e replicação regional
Backup e	Backups	Backups	Backups



Restauração	automáticos, suporte a restauração pontual	automáticos e restauração configurável	automáticos e restauração pontual
Segurança	Integração com IAM, criptografia em trânsito e repouso	Azure Active Directory (AAD), criptografia, detecção de ameaças	IAM do Google Cloud, criptografia em trânsito e repouso
Conformidade	GDPR, HIPAA, PCI DSS, SOC 1/2/3	GDPR, ISO 27001, PCI DSS	GDPR, ISO 27001, PCI DSS
Gerenciamento	Simplificado via Console AWS, suporte ao CLI e SDK	Integrado ao Portal Azure, integração com ferramentas Microsoft	Gerenciado via Console Google Cloud
Modelos de Preços	Sob demanda, instâncias reservadas	Modelos DTU (capacidade fixa) ou vCore (ajustável)	Baseado em minutos de uso
Casos de Uso Comuns	E-commerce, sistemas empresariais, bancos OLAP e OLTP	ERP, aplicações corporativas, migração de SQL Server	Aplicações móveis, SaaS, sistemas distribuídos
Integração com o Ecossistema	Lambda, S3, CloudWatch, Aurora	Power BI, Data Factory, Machine Learning	BigQuery, Looker, Firebase

Fonte: Cloud SQL.

TEMA 3 – BANCO DE DADOS NOSQL: AWS

O Amazon DynamoDB é um serviço de banco de dados NoSQL totalmente gerenciado que oferece desempenho rápido e baixa latência, escalando facilmente conforme necessário. Ele elimina os encargos administrativos de operar um banco de dados NoSQL distribuído, permitindo que nos concentremos na aplicação. O Amazon DynamoDB simplifica significativamente o provisionamento de hardware, configuração e instalação, replicação, aplicação de patches de software e escalonamento de clusters para bancos de dados NoSQL.



O Amazon DynamoDB foi projetado para simplificar o gerenciamento de bancos de dados e clusters, fornecer níveis consistentemente altos de desempenho, facilitar tarefas de escalabilidade e melhorar a confiabilidade com replicação automática. Os desenvolvedores podem criar uma tabela no Amazon DynamoDB e gravar um número ilimitado de itens com latência consistente.

O DynamoDB oferece níveis consistentes de desempenho distribuindo automaticamente os dados e o tráfego de uma tabela em várias partições. Depois de configurar uma capacidade de leitura ou gravação, o DynamoDB adiciona automaticamente a infraestrutura necessária para suportar os níveis de rendimento solicitados. À medida que a demanda muda ao longo do tempo, podemos ajustar a capacidade de leitura ou gravação após a criação de uma tabela, e o DynamoDB adicionará ou removerá infraestrutura e ajustará o particionamento interno de acordo.

Para manter níveis consistentes e rápidos de desempenho, todos os dados das tabelas são armazenados em discos SSD de alto desempenho. Métricas de desempenho, incluindo taxas de transações, podem ser monitoradas usando o Amazon CloudWatch. Além de oferecer alto desempenho, o DynamoDB também garante alta disponibilidade e durabilidade automática, replicando dados em várias zonas de disponibilidade dentro de uma região da AWS.

3.1 Modelo de dados

Os componentes básicos do modelo de dados do Amazon DynamoDB incluem tabelas, itens e atributos. Conforme ilustrado na Figura 2 a seguir, uma tabela é uma coleção de itens, e cada item é uma coleção de um ou mais atributos. Cada item também possui uma chave primária que o identifica de forma única.

Figura 2 – Relação entre tabelas, itens e atributos em Banco de Dados DynamoDB



Fonte: Amazon DynamoDB.



Em um banco de dados relacional, uma tabela possui um esquema predefinido que inclui o nome da tabela, chave primária, lista de nomes das colunas e seus respectivos tipos de dados. Todos os registros armazenados na tabela devem ter o mesmo conjunto de colunas. Em contraste, o Amazon DynamoDB exige apenas que uma tabela tenha uma chave primária, sem a necessidade de definir previamente todos os nomes de atributos e seus tipos de dados. Os itens individuais em uma tabela do Amazon DynamoDB podem ter qualquer número de atributos, embora exista um limite de 400 KB para o tamanho total do item.

Cada atributo em um item é um par nome/valor. Um atributo pode ter um único valor ou ser um conjunto de múltiplos valores. Por exemplo, um item de livro pode ter os atributos “título” e “autores”. Cada livro terá um único título, mas pode ter vários autores. O atributo com múltiplos valores é representado como um conjunto; valores duplicados não são permitidos. Os dados no Amazon DynamoDB são armazenados como pares chave/valor, como no exemplo a seguir.

```
{  
  Id = 101  
  ProductName = "Book 101 Title"  
  ISBN = "123-1234567890"  
  Authors = [ "Author 1", "Author 2" ]  
  Price = 2.88  
  Dimensions = "8.5 x 11.0 x 0.5"  
  PageCount = 500  
  InPublication = 1  
  ProductCategory = "Book"  
}
```

Os aplicativos podem se conectar ao endpoint do serviço Amazon DynamoDB e enviar solicitações via HTTP/S para ler e gravar itens em uma tabela ou até mesmo criar e excluir tabelas. O DynamoDB fornece uma API de serviço web que aceita solicitações no formato JSON. Embora possamos programar diretamente contra os endpoints da API do serviço web, a maioria dos desenvolvedores opta por usar o AWS Software Development Kit (SDK) para



interagir com seus itens e tabelas. O AWS SDK está disponível em várias linguagens e oferece uma interface de programação simplificada e de alto nível.

3.2 Tipos de dados

O Amazon DynamoDB oferece grande flexibilidade para o esquema de banco de dados. Diferentemente de um banco de dados relacional tradicional, que exige que definamos os tipos de coluna com antecedência, o DynamoDB requer apenas o atributo da chave primária. Cada item adicionado à tabela pode incluir atributos adicionais. Isso proporciona flexibilidade para expandir o esquema ao longo do tempo sem precisar reconstruir toda a tabela ou lidar com diferenças de versões de registros na lógica da aplicação.

Ao criar uma tabela ou um índice secundário, é necessário especificar os nomes e os tipos de dados de cada atributo da chave primária (chave de partição e chave de ordenação). O Amazon DynamoDB oferece suporte a uma ampla gama de tipos de dados para atributos, classificados em três categorias principais: Escalar, Conjunto (Set) ou Documento (Document).

3.3 Tipos de dados escalares

Um tipo escalar representa exatamente um valor. O Amazon DynamoDB suporta os seguintes cinco tipos escalares.

- **String:** texto e caracteres de comprimento variável de até 400 KB. Suporta Unicode com codificação UTF-8.
- **Number:** números positivos ou negativos com até 38 dígitos de precisão.
- **Binary:** dados binários, imagens ou objetos compactados de até 400 KB.
- **Boolean:** um valor binário que representa verdadeiro ou falso.
- **Null:** representa um estado em branco, vazio ou desconhecido. String, Number, Binary e Boolean não podem ser vazios.

3.4 Tipos de dados de conjunto (Set)

Os conjuntos são úteis para representar uma lista única de um ou mais valores escalares. Cada valor em um conjunto deve ser único e ter o mesmo tipo de dado. Conjuntos não garantem ordem. O Amazon DynamoDB suporta três tipos de conjuntos:



- String Set – lista única de atributos String;
- Number Set – lista única de atributos Number;
- Binary Set – lista única de atributos Binary.

3.5 Tipos de dados de documento

O tipo de documento é útil para representar múltiplos atributos aninhados, semelhante à estrutura de um arquivo JSON. O Amazon DynamoDB suporta dois tipos de documento:

- List – cada lista pode ser usada para armazenar uma lista ordenada de atributos de diferentes tipos de dados;
- Map – cada mapa pode ser usado para armazenar uma lista não ordenada de pares chave/valor. Mapas podem representar a estrutura de qualquer objeto JSON.

3.6 Chave primária

Ao criar uma tabela, é necessário especificar a chave primária além do nome da tabela. Assim como em um banco de dados relacional, a chave primária identifica de forma única cada item na tabela. Uma chave primária sempre apontará para exatamente um item. O Amazon DynamoDB oferece suporte a dois tipos de chaves primárias, e essa configuração não pode ser alterada após a criação da tabela.

3.6.1 Chave de Partição (Partition Key)

A chave primária é composta por um único atributo, conhecido como “chave de partição” (ou “chave de hash”).

O DynamoDB constrói um índice de hash não ordenado com base nesse atributo de chave primária.

3.6.2 Chave de Partição e Chave de Ordenação (Partition and Sort Key)

A chave primária é composta por dois atributos. O primeiro é a chave de partição e o segundo é a chave de ordenação (ou chave de intervalo).

Cada item na tabela é identificado de forma única pela combinação dos valores da chave de partição e da chave de ordenação. É possível que dois itens



tenham o mesmo valor para a chave de partição, mas esses itens devem ter valores diferentes para a chave de ordenação.

Além disso, cada atributo da chave primária deve ser definido como do tipo string, number ou binary. O Amazon DynamoDB usa a chave de partição para direcionar a solicitação para a partição correta no sistema.

3.7 Capacidade de provisionamento

Ao criar uma tabela no Amazon DynamoDB, é necessário provisionar uma quantidade específica de capacidade de leitura e gravação para atender às cargas de trabalho esperadas. Com base nas configurações fornecidas, o DynamoDB provisiona a infraestrutura necessária para atender às suas necessidades, garantindo tempos de resposta consistentes e de baixa latência. A capacidade geral é medida em unidades de capacidade de leitura e gravação. Esses valores podem ser ajustados posteriormente utilizando a ação `UpdateTable`.

Cada operação realizada em uma tabela do Amazon DynamoDB consome parte das unidades de capacidade provisionadas. A quantidade consumida depende, principalmente, do tamanho do item, mas também de outros fatores. Para operações de leitura, o consumo também é influenciado pela consistência da leitura configurada na solicitação (eventual ou forte).

3.7.1 Exemplo de consumo de capacidade

Para uma tabela sem índice secundário local, ler um item de até 4 KB consome 1 unidade de capacidade de leitura.

Para operações de gravação, gravar um item de até 1 KB consome 1 unidade de capacidade de gravação.

Ler um item de 110 KB consumirá 28 unidades de capacidade de leitura ($110 / 4 = 27,5$, arredondado para cima). Para leituras com consistência forte, esse número é dobrado, totalizando 56 unidades de capacidade.

Podemos utilizar o Amazon CloudWatch para monitorar a capacidade do DynamoDB e tomar decisões de escalonamento. Métricas como `ConsumedReadCapacityUnits` e `ConsumedWriteCapacityUnits` permitem acompanhar o consumo. Caso a capacidade provisionada seja excedida por um período, as solicitações serão limitadas (throttling) e poderão ser retomadas



posteriormente. É possível configurar alertas com a métrica `ThrottledRequests` no Amazon CloudWatch para identificar mudanças no uso.

3.8 Índices secundários

Ao criar uma tabela com uma chave de partição e uma chave de ordenação (anteriormente chamadas de hash e range key), é possível definir opcionalmente um ou mais índices secundários. Os índices secundários permitem consultar os dados na tabela usando uma chave alternativa, além das consultas pela chave primária. O Amazon DynamoDB oferece dois tipos de índices secundários:

- **índice secundário global (Global Secondary Index)** – um índice com uma chave de partição e uma chave de ordenação diferentes das da tabela. Pode ser criado ou excluído a qualquer momento.
- **índice secundário local (Local Secondary Index)** – um índice que utiliza a mesma chave de partição da chave primária da tabela, mas uma chave de ordenação diferente. Só pode ser criado durante a criação da tabela.

Os índices secundários permitem buscar em tabelas grandes de forma eficiente, evitando operações de escaneamento dispendiosas para encontrar itens com atributos específicos. Eles oferecem suporte a padrões de acesso e casos de uso diferentes do que seria possível apenas com uma chave primária. Enquanto uma tabela pode ter apenas um índice secundário local, é possível criar múltiplos índices secundários globais.

3.8.1 Consumo de capacidade em índices secundários

O DynamoDB atualiza cada índice secundário quando um item é modificado, consumindo unidades de capacidade de gravação.

Para índices secundários locais, as atualizações consomem unidades de capacidade da tabela principal.

Para índices secundários globais, a capacidade de rendimento é configurada separadamente da tabela.

3.9 Gravação e leitura de dados

Após criar uma tabela com uma chave primária e índices, é possível



começar a gravar e ler itens na tabela. O Amazon DynamoDB oferece diversas operações para criar, atualizar e excluir itens individuais. Também há múltiplas opções de consulta, permitindo pesquisar uma tabela ou índice e recuperar itens específicos ou um lote de itens.

Com essas operações, o DynamoDB suporta uma ampla gama de casos de uso, desde consultas simples até análises mais complexas em grandes volumes de dados.

3.10 Gravando itens

O Amazon DynamoDB fornece três principais ações na API para criar, atualizar e excluir itens: `PutItem`, `UpdateItem` e `DeleteItem`.

3.10.1 PutItem

Permite criar um novo item com um ou mais atributos. Caso a chave primária já exista, a chamada ao `PutItem` atualizará o item existente. Requer apenas o nome da tabela e a chave primária; atributos adicionais são opcionais.

3.10.2 UpdateItem

Localiza itens existentes com base na chave primária e substitui os atributos especificados. Permite atualizar apenas um único atributo, mantendo os outros inalterados. Também pode ser usado para criar itens caso eles ainda não existam. Suporta contadores atômicos, que garantem consistência em operações simultâneas para incrementar ou decrementar valores.

3.10.3 DeleteItem

Remove um item de uma tabela com base na chave primária especificada. Essas ações suportam expressões condicionais, permitindo validar condições antes de aplicar a ação. Por exemplo, é possível usar uma expressão condicional no `PutItem` para evitar sobrescritas acidentais ou aplicar validações de lógica de negócios.

3.11 Lendo itens

Após criar um item, ele pode ser recuperado de duas maneiras principais:



GetItem: recupera um item com base em sua chave primária. Por padrão, retorna todos os atributos do item, mas é possível filtrar atributos específicos.

Se a chave primária for composta, é necessário especificar tanto a chave de partição quanto a chave de ordenação.

Cada chamada ao GetItem consome unidades de capacidade de leitura com base no tamanho do item e na consistência da leitura selecionada.

3.12 Consistência da leitura

- Leitura eventualmente consistente (padrão): pode não refletir imediatamente as alterações de uma operação de escrita recente.
- Leitura fortemente consistente: garante a versão mais atualizada do item, mas consome mais capacidade de leitura e pode ser menos disponível em caso de atrasos na rede.

3.13 Consistência eventual

O DynamoDB é um sistema distribuído que replica itens em vários servidores dentro de uma região AWS para alta disponibilidade e durabilidade. Quando um item é atualizado, a replicação pode levar algum tempo para completar, resultando em consistência eventual. Isso significa que:

- uma leitura logo após uma gravação pode não refletir a alteração mais recente;
- a consistência entre todas as cópias geralmente é alcançada em cerca de um segundo.

3.14 Leitura eventualmente consistente

A resposta pode incluir dados desatualizados.

Repetir a solicitação após um curto período geralmente retorna os dados mais recentes.

3.15 Leitura fortemente consistente

- Garante que a leitura reflita as atualizações de todas as operações de gravação bem-sucedidas anteriores.
- Pode ser menos disponível em caso de problemas de rede.



3.16 Operações em lote

O Amazon DynamoDB também oferece operações para lidar com grandes lotes de itens.

- **BatchWriteItem:** permite realizar até 25 operações de criação ou atualização de itens em uma única solicitação, minimizando o overhead.
- **BatchGetItem:** permite recuperar vários itens em uma única operação.

3.17 Buscando itens

O DynamoDB fornece duas operações principais para buscar itens em uma tabela ou índice: Query e Scan.

- Query:
 - Permite encontrar itens usando valores dos atributos de chave primária em uma tabela ou índice secundário.
 - Requer um nome de atributo de chave de partição e um valor distinto para a busca.
 - Opcionalmente, é possível fornecer um valor de chave de ordenação e usar operadores de comparação para refinar os resultados.
 - Os resultados são ordenados automaticamente pela chave primária e limitados a 1 MB.
- Scan:
 - Lê todos os itens de uma tabela ou índice secundário.
 - Retorna todos os atributos dos itens por padrão, com limite de 1 MB por solicitação.
 - Permite filtrar itens usando expressões, mas pode ser uma operação intensiva em recursos.
 - Caso o conjunto de resultados de um Query ou Scan exceda 1 MB, os resultados podem ser paginados em incrementos de 1 MB.

Essas operações fornecem flexibilidade para atender a diferentes padrões de consulta e casos de uso.



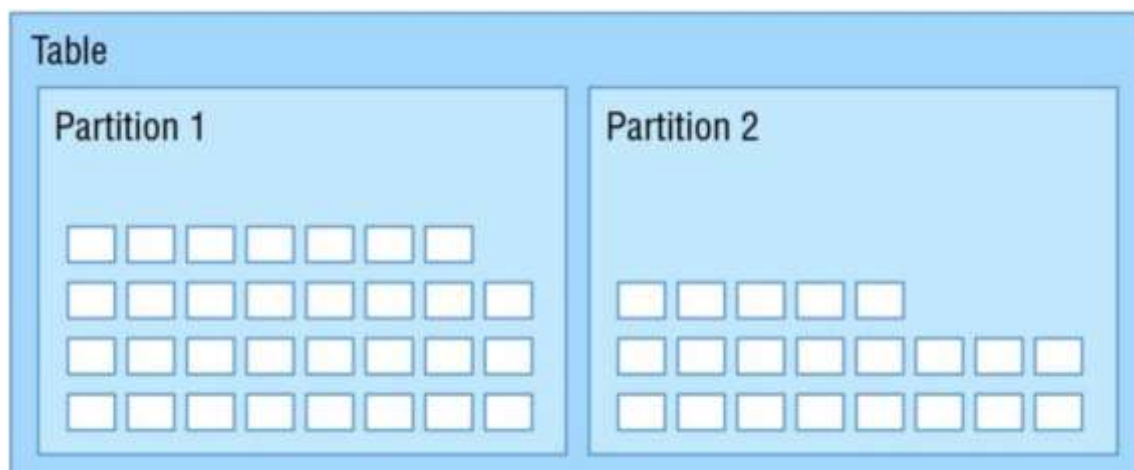
3.21 Escalabilidade e particionamento

O Amazon DynamoDB é um serviço totalmente gerenciado que abstrai a maior parte da complexidade envolvida na construção e escalabilidade de um cluster NoSQL. Ele permite a criação de tabelas que podem escalar para armazenar virtualmente um número ilimitado de itens, mantendo um desempenho consistente e de baixa latência.

Uma tabela no Amazon DynamoDB pode escalar horizontalmente utilizando partições para atender às necessidades de armazenamento e desempenho da sua aplicação. Cada partição individual representa uma unidade de capacidade de computação e armazenamento. Uma aplicação bem projetada leva em conta a estrutura de particionamento da tabela para distribuir uniformemente as transações de leitura e gravação, alcançando altas taxas de transação com baixa latência.

Os itens de uma tabela no DynamoDB são armazenados em várias partições. O DynamoDB decide em qual partição armazenar o item com base na chave de partição. A chave de partição é usada para distribuir o novo item entre todas as partições disponíveis, garantindo que itens com a mesma chave de partição sejam armazenados na mesma partição.

Figura 3 – Particionamento de tabelas



Fonte: Amazon DynamoDB.

À medida que o número de itens em uma tabela cresce, novas partições podem ser adicionadas ao dividir uma partição existente. O throughput provisionado configurado para a tabela é dividido igualmente entre as partições.



A capacidade provisionada alocada para uma partição é completamente dedicada a ela, sem compartilhamento entre partições.

Ao criar uma tabela, o Amazon DynamoDB configura as partições com base na capacidade de leitura e gravação desejada. Cada partição individual pode armazenar cerca de 10 GB de dados e suporta no máximo:

- 3.000 unidades de capacidade de leitura; ou
- 1.000 unidades de capacidade de gravação.

Para partições que não utilizam completamente sua capacidade provisionada, o DynamoDB fornece capacidade de burst (explosão) para lidar com picos de tráfego. Uma parte da capacidade não utilizada é reservada para lidar com esses picos por curtos períodos.

3.22 Divisão de partições

À medida que as necessidades de armazenamento ou capacidade aumentam, o DynamoDB pode dividir uma partição para acomodar mais dados ou taxas maiores de solicitações provisionadas. No entanto, uma vez que uma partição é dividida, ela não pode ser mesclada novamente. Isso deve ser levado em consideração ao planejar um aumento temporário na capacidade provisionada, seguido de uma redução. Com cada nova partição adicionada, a parte da capacidade provisionada atribuída a cada partição é reduzida.

3.23 Distribuição uniforme de trabalho

Para alcançar a capacidade total de solicitações provisionadas de uma tabela, é essencial distribuir uniformemente a carga de trabalho entre os valores da chave de partição. Isso garante que as solicitações sejam distribuídas entre as várias partições.

Por exemplo, se uma tabela tem 10.000 unidades de capacidade de leitura configuradas, mas todo o tráfego é direcionado a uma única chave de partição, não será possível utilizar mais do que o limite máximo de 3.000 unidades de capacidade de leitura que uma partição pode suportar.

Distribuir as solicitações entre as chaves de partição permite utilizar de forma eficiente toda a capacidade provisionada da tabela.



3.24 Segurança

O Amazon DynamoDB oferece controle granular sobre os direitos de acesso e permissões para usuários e administradores. Ele se integra ao serviço Identity and Access Management (IAM) para fornecer controle rigoroso de permissões por meio de políticas. É possível criar uma ou mais políticas que permitem ou negam operações específicas em tabelas específicas. Também é possível usar condições para restringir o acesso a itens ou atributos individuais.

Todas as operações devem ser autenticadas como um usuário ou sessão de usuário válidos. Aplicações que precisam ler e gravar dados no DynamoDB devem obter um conjunto de chaves de controle de acesso temporárias ou permanentes. Embora essas chaves possam ser armazenadas em arquivos de configuração, a melhor prática é que aplicações executadas na AWS usem perfis de instância IAM Amazon EC2 para gerenciar credenciais.

Os perfis de instância IAM Amazon EC2 ou funções permitem evitar o armazenamento de chaves sensíveis em arquivos de configuração, eliminando a necessidade de protegê-los separadamente. Essa abordagem melhora a segurança e reduz o risco de exposição de credenciais.

O Amazon DynamoDB também oferece suporte para controle de acesso detalhado, que pode restringir o acesso a itens específicos dentro de uma tabela ou até mesmo a atributos específicos dentro de um item. Por exemplo, é possível limitar um usuário a acessar apenas os itens relacionados a ele em uma tabela e impedir o acesso a itens associados a outros usuários.

Usando condições em uma política do IAM, é possível restringir as ações que um usuário pode executar, em quais tabelas ele pode operar e quais atributos ele pode ler ou gravar. Isso fornece um nível adicional de controle para proteger dados sensíveis e garantir que os usuários tenham acesso apenas às informações que lhes são permitidas.

TEMA 4 – BANCO DE DADOS NOSQL: OUTROS PROVEDORES

Os bancos de dados NoSQL oferecidos por provedores como Azure e Google Cloud são projetados para atender às necessidades de aplicações modernas que demandam alta escalabilidade, flexibilidade no esquema de dados e desempenho otimizado para grandes volumes de informações. Diferentemente dos bancos relacionais, os bancos NoSQL não exigem



esquemas rígidos, permitindo a modelagem de dados com mais flexibilidade e eficiência.

As categorias principais de bancos de dados NoSQL incluem:

- chave-valor – projetados para operações rápidas e simples. Exemplo: Azure Table Storage;
- documentos – armazenam dados semiestruturados em formato JSON ou BSON. Exemplo: Google Firestore;
- colunar – ideais para armazenar grandes volumes de dados organizados por colunas, como logs. Exemplo: Google Bigtable;
- grafos – utilizados para modelar relações complexas entre entidades. Exemplo: Azure Cosmos DB com suporte a Gremlin.

Esses modelos são implementados de forma personalizada por cada provedor, otimizando o desempenho e a integração com seus respectivos ecossistemas.

4.1 Azure Cosmos DB

O Azure Cosmos DB é um banco de dados multimodelo totalmente gerenciado pela Microsoft, projetado para oferecer alta disponibilidade, desempenho global e flexibilidade no armazenamento de dados.

- **Características principais:**
 - **Multimodelo:** suporte a diferentes modelos de dados, incluindo chave-valor, documentos, grafos e tabelas colunares.
 - **Replicação global:** replica dados automaticamente em várias regiões, garantindo baixa latência e alta disponibilidade.
 - **Consistência configurável:** oferece níveis ajustáveis de consistência, incluindo consistência forte, eventual e modelos intermediários.
 - **Escalabilidade elástica:** permite escalabilidade horizontal com ajuste automático de capacidade.
- **Casos de uso:**
 - Aplicações globais distribuídas.
 - Processamento de grandes volumes de transações em tempo real.
 - Sistemas de recomendação e análise preditiva.



4.2 Google Firestore

O Google Firestore é um banco de dados NoSQL baseado em documentos, projetado para aplicações móveis e web que exigem sincronização em tempo real e escalabilidade automática.

- **Características principais:**

- Modelo de documentos: estrutura de dados baseada em JSON para armazenar informações semiestruturadas.
- Integração com Firebase: suporte nativo para sincronização em tempo real em dispositivos móveis e web.
- Escalabilidade automática: adapta-se automaticamente às mudanças na carga de trabalho, reduzindo a necessidade de gerenciamento manual.
- Alta disponibilidade: dados replicados em múltiplas zonas dentro de uma região.

- **Casos de uso:**

- Aplicações móveis e web com atualizações em tempo real.
- Chats, sistemas de colaboração e redes sociais.
- Armazenamento de dados para aplicativos baseados em Firebase.

No Quadro 3 a seguir, comparamos os serviços de bancos de dados NoSQL entre os provedores (AWS, Azure e GCP).

Quadro 3 – Comparação entre os Bancos NoSQL entre os provedores

Aspecto	Amazon Dynamo DB	Azure Cosmos DB	Google Firestore
Modelo de Dados	Chave-Valor, Documentos	Multimodelos (Documentos, Grafos)	Documentos
Escalabilidade	Automática com particionamento	Global com replicação e ajuste elástico	Automática baseada em carga
Integração	Serviços AWS como Lambda e S3	Multi-Regiões com consistência ajustável	Multi-Regiões com alta disponibilidade
Replicação	IoT, catálogos e	Sistemas globais,	Aplicações

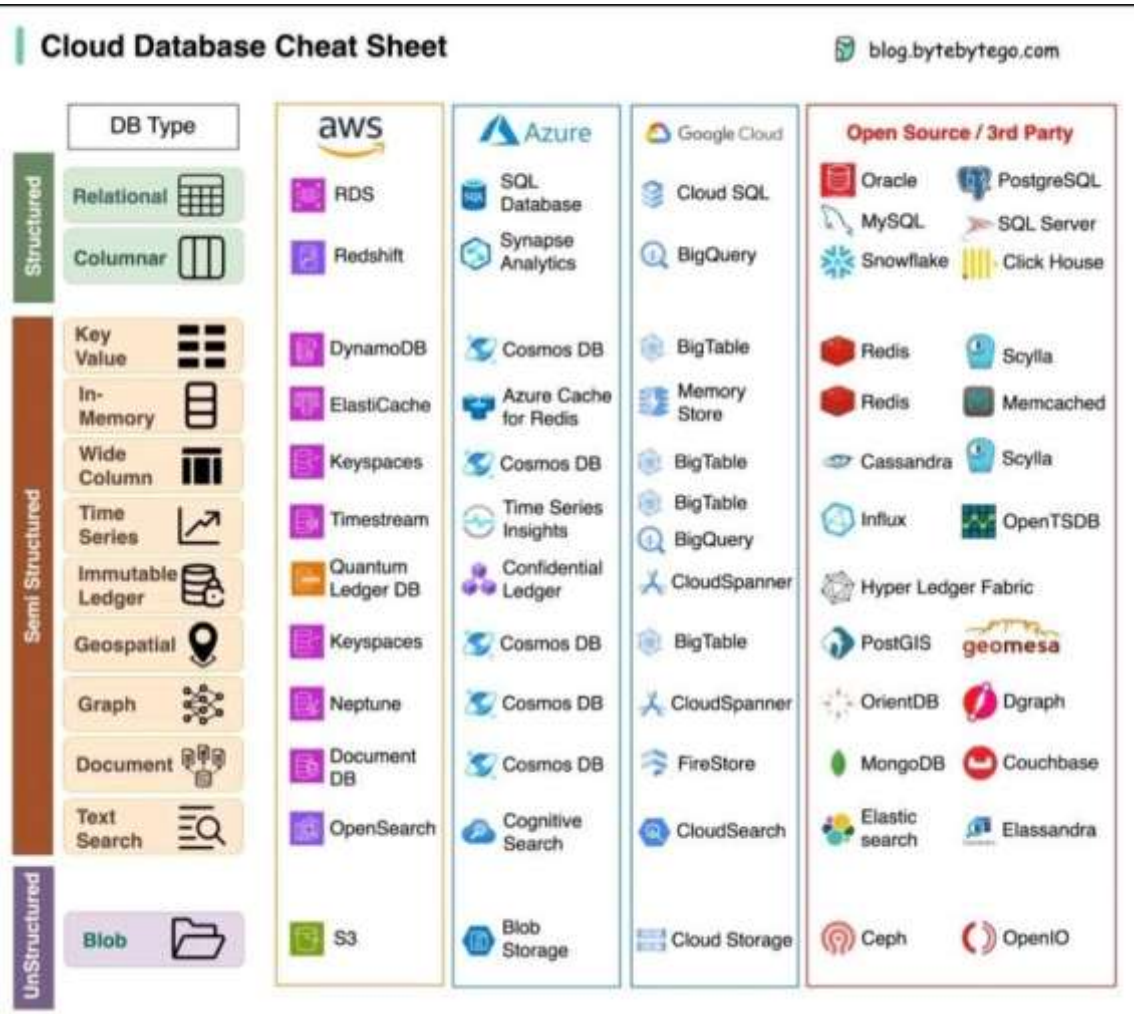


	e-commerce	análises em tempo real	móveis e colaboração
--	------------	------------------------	----------------------

Fonte: Maike Cristian Rebelo de Lima.

Na Figura 4 a seguir, comparamos os tipos de Banco de Dados (SQL e NoSQL) entre os provedores.

Figura 3 – Comparação entre os Bancos SQL e NoSQL nos provedores



Fonte: Byte Byte Go, 2023.

TEMA 5 – BANCOS DE DADOS NEWSQL

Os bancos de dados NewSQL surgiram como uma evolução dos bancos de dados tradicionais, combinando as vantagens dos bancos de dados relacionais (SQL) com os benefícios de escalabilidade e desempenho dos bancos NoSQL. Eles foram projetados para atender às demandas de aplicativos modernos que requerem alta consistência, baixa latência e escalabilidade



horizontal, sem abrir mão do suporte a transações Acid (Atomicidade, Consistência, Isolamento, Durabilidade).

5.1 O que é NewSQL?

- Origem do termo: foi cunhado para descrever sistemas que oferecem o modelo relacional de dados e suporte para SQL, mas com a capacidade de escalar horizontalmente em ambientes distribuídos, algo que tradicionalmente era uma limitação dos bancos de dados relacionais.
- Objetivo principal: resolver as limitações de desempenho e escalabilidade dos bancos SQL tradicionais sem sacrificar transações consistentes e complexas.

5.2 Características principais

1. **Modelo relacional com suporte a SQL:** assim como os bancos relacionais tradicionais, os bancos NewSQL utilizam tabelas, chaves primárias e SQL para manipulação de dados.
2. **Escalabilidade horizontal:** diferentemente dos bancos SQL tradicionais, que geralmente escalam verticalmente (aumentando recursos do servidor), os bancos NewSQL suportam a distribuição de dados e operações em múltiplos servidores.
3. **Transações Acid:** garantem a consistência e integridade dos dados, mesmo em ambientes distribuídos.
4. **Baixa latência:** projetados para lidar com grandes volumes de consultas em tempo real.
5. **Arquitetura distribuída:** dados e processamento são distribuídos em múltiplos nós, garantindo alta disponibilidade e tolerância a falhas.

5.3 Exemplos de bancos de dados NewSQL

1. CockroachDB:
 - Inspirado no Google Spanner, é um banco de dados distribuído que oferece escalabilidade horizontal e transações Acid.
 - Recursos:
 - Alta disponibilidade com replicação automática de dados.
 - Compatibilidade com SQL padrão.



- Ideal para aplicativos globais que requerem consistência forte.
- Casos de uso:
 - Sistemas financeiros, comércio eletrônico e gerenciamento de usuários em escala global.

2. Google Spanner:

- Banco de dados distribuído do Google que combina consistência forte com escalabilidade global.
- Recursos:
 - Suporte a transações distribuídas.
 - Baixa latência para leituras e gravações globais.
- Casos de uso:
 - Aplicações corporativas globais, como CRMs e ERPs.

3. YugaByteDB:

- Banco de dados NewSQL distribuído que suporta tanto APIs SQL quanto NoSQL.
- Recursos:
 - Suporte a PostgreSQL e Cassandra.
 - Ideal para aplicações que exigem alta consistência e baixa latência.
- Casos de uso:
 - Sistemas híbridos que combinam transações complexas e operações simples de chave-valor.

4. NuoDB:

- Um banco de dados relacional projetado para ambientes em nuvem.
- Recursos:
 - Escalabilidade elástica.
 - Projetado para nuvens públicas e privadas.
- Casos de uso:
 - Aplicações SaaS e migração de sistemas legados para a nuvem.

No Quadro 4 a seguir, comparamos as características dos bancos de dados que estamos abordando (SQL, NoSQL e NewSQL).



Quadro 4 – Comparativo entre os tipo de Banco de Dados SQL, NoSQL e NewSQL

Aspecto	SQL	NoSQL	NewSQL
Modelo de Dados	Relacional	Não-Relacional	Relacional
Escalabilidade	Vertical	Horizontal	Horizontal
Transações	ACID	Geralmente BASE	ACID
Uso Principal	Sistemas Tradicionais, ERP	Big Data, IoT, Redes Sociais	Aplicações Transacionais Modernas

Fonte: Maike Cristian Rebelo de Lima.

FINALIZANDO

Nesta abordagem, exploramos as principais categorias de bancos de dados modernos, incluindo SQL, NoSQL e NewSQL, destacando suas características, diferenças e aplicações práticas. A evolução das tecnologias de banco de dados reflete a necessidade crescente de lidar com volumes massivos de dados, escalabilidade global, e alta disponibilidade em um cenário cada vez mais distribuído.

Os bancos de dados relacionais (SQL) continuam sendo a escolha principal para sistemas que exigem consistência, suporte a transações complexas e um esquema bem definido. Por outro lado, os bancos NoSQL oferecem flexibilidade e desempenho otimizado para aplicações que demandam modelos de dados mais dinâmicos e escalabilidade horizontal. Por fim, os bancos NewSQL surgem como uma ponte entre esses dois mundos, entregando escalabilidade distribuída sem comprometer as garantias transacionais.

À medida que as organizações enfrentam desafios mais complexos, como a necessidade de processar dados em tempo real, suportar transações globais e integrar diferentes fontes de dados, a escolha da tecnologia de banco de dados ideal torna-se um fator estratégico. Não há uma solução única para todos os cenários, mas compreender os pontos fortes e limitações de cada abordagem é essencial para arquitetar sistemas eficientes e resilientes.

Independentemente da escolha da tecnologia, a integração com os ecossistemas de nuvem de provedores como AWS, Azure e Google Cloud



permite que empresas implementem soluções escaláveis e seguras, otimizando custos e melhorando o desempenho. O domínio dessas ferramentas e práticas garante que profissionais estejam bem-preparados para atender às demandas do mercado e suportar os sistemas críticos do futuro.

Assim, este conteúdo fornece as bases para compreender o papel dos bancos de dados na era da computação em nuvem, promovendo a reflexão sobre como melhor utilizar essas tecnologias para resolver problemas reais. Com isso, encerramos este tópico, convidando você a explorar mais a fundo cada uma das tecnologias apresentadas, buscando alinhar as capacidades dos sistemas com as necessidades específicas de cada aplicação.



REFERÊNCIAS

AMAZON WEB SERVICES – AWS. **Amazon DynamoDB Developer Guide**. Disponível em: <<https://docs.aws.amazon.com/amazondynamodb/index.html>>. Acesso em: 10 jan. 2025.

_____. **Amazon DocumentDB Developer Guide**. Disponível em: <<https://docs.aws.amazon.com/documentdb/index.html>>. Acesso em: 10 jan. 2025.

CODD, E. F. Is your DBMS really relational? **Computerworld**, Oct 14, 1985, v. 19, p. ID1.

_____. Does your DBMS run by the rules? **Computerworld**, Oct 21, 1985, v. 19, p. 49.

EP73: Cheat sheet of different databases in cloud services. **ByteByteGo**, Aug 19, 2023. Disponível em: <<https://blog.bytebytego.com/p/ep73-cheat-sheet-of-different-databases>>. Acesso em: 10 jan. 2025.

GOOGLE CLOUD. **Google Firestore Documentation**. Disponível em: <<https://firebase.google.com/docs/firestore>>. Acesso em: 10 jan. 2025.

_____. **Google Spanner Documentation**. Disponível em: <<https://cloud.google.com/spanner>>. Acesso em: 10 jan. 2025.

MICROSOFT AZURE. **Azure Cosmos DB Documentation**. Disponível em: <<https://learn.microsoft.com/en-us/azure/cosmos-db/>>. Acesso em: 10 jan. 2025.
